Trainer Teaching Guide: Spring Boot Session 3 - DTO, Layers & Exception Handling

A detailed, textbook-style script for springboot-session3-theory.html. Simple English, examples, analogies and natural fillers - read it aloud and it sounds like teaching, not reading.

Presenter: Purvam Prajapati - Faculty, AHD
Companion to: springboot-session3-theory.html (21 slides)

HOW TO USE THIS GUIDE:
- ON SCREEN tells you what the slide shows.
- SAY is the full teaching script - read it word-for-word; fillers and questions are built in. Pause where you see '...'.
- EMPHASISE / DELIVERY TIP are notes for you - do NOT read these aloud.

Opening

Slide 1 - Title: DTO, Layered Architecture & Exception Handling

ON SCREEN

The blue cover slide naming the session, plus 'a first look at Spring Security and Microservices'.

SAY (read this aloud)

Welcome back, everyone. Let me set the scene for today. In Session 2, we built a working API - it could create, read, update, and delete students. It worked. But, between you and me, it was a bit rough around the edges. Today, we take that rough, working code and make it professional - the kind of code you would be proud to ship at a real company.

We will do three things. First, we will organise our code into clean layers, so each part has one clear job. Second, we will stop exposing our raw database objects to the outside world - we will use something called a DTO. And third, we will handle errors gracefully, so that when something goes wrong, the caller gets a clear, friendly message instead of an ugly crash.

And then, to finish, we will take a first peek at two big topics - Spring Security and microservices - just to see the road ahead. So today is really about turning a beginner's app into a grown-up one. Let's go.

DELIVERY TIP

Frame the whole session as 'making working code professional'. That narrative - rough to polished - gives the three topics a single, motivating thread.

Slide 2 - Agenda

ON SCREEN

Six agenda items: layered architecture, DTO, exception handling, Spring Security demo, microservices concepts, recap.

SAY (read this aloud)

Here is our plan. First, layered architecture - the idea of Controller, Service, and Repository, each doing one job. Then DTOs - and crucially, why you should never expose your database object directly. Then exception handling - turning ugly crashes into clean 404 messages. Then a short, five-minute demo of Spring Security - just watching, no coding. Then the big-picture concepts of microservices. And a recap with a handy annotation cheat-sheet at the end.

So the thread running through all of it is the same: we are taking the working-but-rough API from last time and making it clean, safe, and professional. Let's start with the first problem.

Slide 3 - Session 2 recap

ON SCREEN

Three click-to-reveal flash cards: @RequestBody, POST vs PUT, and @PathVariable.

SAY (read this aloud)

Quick warm-up, three questions - answer in your head, then we reveal.

Question one: what does at-RequestBody do? Think... Reveal. It takes the JSON sent in the request body and turns it into a Java object for you, automatically. You put it on a method parameter, and Spring hands you a ready-filled object.

Question two: what is the difference between POST and PUT? Think... Reveal. POST creates a brand-new thing - that is the C in CRUD. PUT updates something that already exists, usually found by an id in the address - that is the U.

And question three: what does at-PathVariable do? Reveal. It pulls a value straight out of the URL path. In slash posts slash five, the 'five' gets handed to your method as the id.

All comfortable? Lovely. Notice today we are switching from students to posts - blog posts - as our example, but every idea is the same.

Part 1 - DTO & Layered Architecture

Slide 4 - What's wrong with returning entities directly?

ON SCREEN

Two code panels: returning a User entity that leaks password and internal flags, versus a UserDTO exposing only safe fields.

SAY (read this aloud)

Let's start with a problem you might not even realise you have. In Session 2, our controller returned the very same object that maps to the database. It was convenient - but it is actually dangerous. Let me show you why.

Look at the left panel. We return a User object straight from the database. But think about what is on a real User record - the name and email, sure, but also the password, an 'is-admin' flag, maybe a count of failed login attempts. And when you return that whole object, ALL of it gets sent out to the caller - including the password. You have just leaked sensitive data to the outside world without meaning to. Yikes.

Now the right panel. Here we return a UserDTO - a separate object that contains only the safe fields: name, email, role. The password and the internal flags simply are not in it, so they cannot leak.

Here is the core problem in one sentence. Your database table and your public response are two different things, with two different audiences. When you let one be the other, you tie them together - and you accidentally expose things you never intended. The fix is a dedicated object just for sending data out. That object is called a DTO.

EMPHASISE

This 'password leak' example is visceral - pause on it. Ask the room: 'would you want your password in the API response?' That makes the need for a DTO obvious and memorable, rather than abstract.

Slide 5 - What is a DTO?

ON SCREEN

A definition and three reason cards: security, flexibility, decoupling.

SAY (read this aloud)

So, what is a DTO? It stands for Data Transfer Object, and it is exactly what the name says - a plain, simple Java class whose only job is to carry data between your app and the outside world. It has no database annotations and no logic. It carries only what the caller actually needs. Let me give you the three reasons we use one, shown as the three cards.

First, security. You hide sensitive fields. The password lives on the entity, but it is simply not on the DTO, so it can never slip out. We just saw that.

Second, flexibility. You shape the response to fit the caller. Maybe one screen needs a short summary and another needs full detail - you can have different DTOs for each, from the same entity. You are not stuck sending every field every time.

Third, decoupling - a fancy word for 'keeping things independent'. Your database can change without breaking your public API. Rename a column, split a table - you just adjust the mapping inside, and the callers outside never even notice.

EMPHASISE

Give them the one-liner to memorise: 'The entity maps to the database. The DTO travels over the wire.' They often look almost identical, but they exist for completely different reasons - and keeping them separate is what lets an API grow safely over years.

Slide 6 - Layered architecture

ON SCREEN

A vertical pipeline: Client, Controller, Service, Repository, Database, each labelled with its annotation and responsibility.

SAY (read this aloud)

This slide is the backbone of the whole session, so let's go slowly. A professional Spring Boot app is split into layers, and each layer has exactly one job. A request flows down through them, and the response flows back up. Let me walk you down the pipeline.

The Client - a browser, a phone, a testing tool - sends an HTTP request. It arrives at the Controller, marked at-RestController. The controller's only job is to deal with the web - take the request, and pass a DTO down to the service. Then the Service, marked at-Service, holds the business logic - the actual thinking - and it does the mapping between DTO and entity. The service calls the Repository, marked at-Repository, whose only job is talking to the database using the entity. And finally the Database stores and retrieves the real data.

Think of it like a restaurant. The waiter is the controller - takes your order, never cooks. The chef is the service - does the real work, never touches the cash register. And the storeroom keeper is the repository - just fetches ingredients. Everyone has one job.

EMPHASISE

Drive home single responsibility: the controller only knows the web, the service only knows business rules, the repository only knows the database. No layer reaches past its neighbour. That discipline is exactly what stops a growing codebase from turning into tangled spaghetti.

Slide 7 - Entity vs DTO, side by side

ON SCREEN

A comparison table, then two code blocks: Post.java the entity and PostDTO.java the DTO.

SAY (read this aloud)

Let me put the entity and the DTO right next to each other so the difference is crystal clear. Read the table with me.

Where does each one live? The entity is mapped to a database table; the DTO just travels between the caller and the service. What annotations? The entity has database annotations - at-Entity, at-Id, and so on; the DTO has none of those - it is just a plain Java class with fields, getters and setters. Who sees it? The entity is internal, never sent out; the DTO is external-facing, it is what callers actually see. And what does it contain? The entity has every column, even the sensitive ones; the DTO has only what the caller needs.

Now look at the code below the table. Post - the entity - carries those database annotations and maps to the posts table. PostDTO has the same four fields, but no database annotations at all. And yes - at this stage they look almost like twins. That is fine and expected. They are twins with different jobs: one works inside with the database, the other goes out into the world. Same fields today, but very different responsibilities as the app grows.

Slide 8 - DTO mapping, the conversion

ON SCREEN

Two helper methods, mapToDTO and mapToEntity, and a round-trip flow diagram for incoming and outgoing data.

SAY (read this aloud)

So if the controller speaks in DTOs and the repository speaks in entities, something has to translate between the two. That something is the service, and it uses two little helper methods. They sound technical, but they are dead simple - they just copy fields across.

The first, mapToDTO, takes an entity and copies its fields into a new DTO. That is the way out - towards the caller. The second, mapToEntity, takes a DTO and copies its fields into a new entity. That is the way in - from the caller towards the database.

Follow the round-trip on screen. Coming in: JSON arrives, becomes a DTO, the service maps it to an entity, and it is saved to the database. Going out: an entity comes from the database, the service maps it to a DTO, and that becomes the JSON response. In and out, with a translation step

each way.

Now, we write these by hand today, on purpose, so you can SEE exactly which fields cross the boundary. In big real projects, teams use libraries like MapStruct to generate this copying automatically - but the idea is identical. It is like a translator at a meeting - same conversation, just converted between two languages each way.

Slide 9 - Getters, Setters & Constructors

ON SCREEN

Two code panels - a plain class with private fields, then the same class with public getters and setters added - plus a note on generating them from the IDE menu.

SAY (read this aloud)

Both of our classes - the entity and the DTO - are plain Java classes with private fields. And private means the outside world cannot touch those fields directly. So for anything else to read or write them, we give each field a small pair of methods - a getter that returns the value, and a setter that changes it. That is the standard boilerplate every Java data class needs.

Why do we care so much? Two reasons. First, Jackson - the library that turns objects into JSON and back - works entirely through these methods. When it builds the response it calls getTitle to read the value for the 'title' key; when a request comes in it calls setTitle to fill the object. Second, JPA needs a no-argument constructor plus the same getters and setters to load and save your entity. No getters and setters, no working API - it really is that fundamental.

Now here is the good news - you do not type all of these by hand, one by one. Every IDE generates them for you. In IntelliJ, press Alt and Insert, choose Getter and Setter, tick all the fields, done. In Eclipse, right-click, Source, then Generate Getters and Setters. In VS Code, right-click and pick Source Action. So four fields become eight methods in a couple of clicks - no extra library, no setup, and it works on any machine.

EMPHASISE

Make everyone actually generate the getters and setters in class - it is the number-one cause of 'my JSON comes back empty' later in the lab. If the getters are missing, Jackson has nothing to read, so the response is blank; if the setters are missing, the incoming data never lands in the object. Generate them once, up front, and the whole lab just works.

Part 2 - Exception Handling

Slide 10 - What happens without custom exception handling?

ON SCREEN

Two mock responses: an ugly 500 with a stack trace, versus a clean 404 with a clear message.

SAY (read this aloud)

Now let's tackle errors. In Session 2, when you asked for a student that did not exist, you got back nothing useful - just an empty result. But if your code actually throws an error, Spring's default response is genuinely ugly. Let me show you the difference, because it is night and day. Look at the left panel - the default. The caller gets a 500 error - which is wrong, because it is not really the server's fault that they asked for a missing item. And worse, it dumps a long, scary stack trace - lines of internal code - right into the response. That leaks your internal details, and it is useless to whoever is calling your API. Imagine getting that as a customer. Confusing and unprofessional.

Now the right panel - what we want. A clean 404 Not Found, with a simple, human message: 'Post not found with id equals five'. Clear, friendly, gives away no internals.

So that right-hand response is our goal for the next few slides. When something is not found, the caller should get a tidy 404 with a helpful message - not a 500 with a wall of red text. And the good news is, it takes just a few lines to get there.

Slide 11 - Custom exception, how it works

ON SCREEN

A three-step visual, then the ResourceNotFoundException class and the orElseThrow line that throws it.

SAY (read this aloud)

So how do we turn that ugly error into a clean one? Just three small steps. Let me walk through them.

Step one - create your own exception class. It extends RuntimeException, and it takes a message in its constructor. So we make a class called ResourceNotFoundException, and we can give it any message we like, such as 'Post not found with id five'.

Step two - and this is the clever bit - we put an annotation on that class: at-ResponseStatus, with NOT FOUND. This single line tells Spring, 'whenever this exception is thrown and nobody catches it, automatically set the response code to 404'. No try-catch needed anywhere. The annotation does the wiring for you.

Step three - we throw it, at the moment we discover the item is missing, using a neat method called orElseThrow, which we will see on the next slide.

Look at the code. The exception class itself is tiny - just the annotation and a constructor that passes the message up. That is it. Three steps - create, annotate, throw - and ugly 500s become clean 404s.

Slide 12 - Optional and orElseThrow()

ON SCREEN

A two-path diagram from findById to present-or-empty, then the orElseThrow code, and a warning about .get().

SAY (read this aloud)

Let me introduce a helpful Java idea called Optional, because it is how we throw that exception cleanly. When you ask the database 'find me post number five' using findById, it does not just hand

you a post or crash. Instead it hands you an Optional - which is like a little box that might contain a post, or might be empty. It is Java's polite way of saying 'maybe it is here, maybe it is not - you must handle both'.

And handling both is easy with one method - orElseThrow. Follow the two paths on screen. findById gives you the box. If the post is inside, orElseThrow opens the box and hands it to you. If the box is empty, orElseThrow throws your ResourceNotFoundException - which becomes the clean 404. Read the code out loud and it almost makes sense in plain English: 'find by id, or else throw this exception'.

So with one line, you handle the happy case and the missing case, no messy null checks needed.

EMPHASISE

Warn about the common mistake in the red box: never call dot get on an Optional directly. If the box is empty, dot get throws a generic, unhelpful error with no message. Always use orElseThrow, so YOU control both the exception and the friendly message the caller sees.

Part 3 - Spring Security & Microservices (overview)

Slide 13 - Spring Security: what it gives you

ON SCREEN

Three cards - blocks everything, login page, generated password - plus the three override properties.

SAY (read this aloud)

Now a first taste of security - and this is a demo, just watching, no coding from you. Here is something that surprises everyone. The moment you add one dependency - Spring Security - to your project, your entire app gets locked down automatically, before you write a single line of security code. Let me show you the three things that happen instantly, on the cards.

First, it blocks everything - every endpoint now demands a login. Hit any URL and you are stopped. Second, it gives you a login page in the browser, for free - you did not build it. The default username is just 'user'. And third, it prints a random password in your console when the app starts, which you use to log in the first time.

And to set your own username and password, you just add three lines to application dot properties. That is it. So with literally one dependency, your app goes from wide open to locked down. Pretty powerful for so little effort.

EMPHASISE

Set expectations clearly, as the slide says: this is a demo walkthrough only, no lab exercise. The full story - JWT tokens, roles, custom config - is a whole dedicated session later. Today's takeaway is just the wow-factor: how little it takes to switch protection on.

Slide 14 - Monolithic architecture: the problem

ON SCREEN

A diagram of one big application with all modules deployed together, and three pain-point cards.

SAY (read this aloud)

To understand microservices, we first need to understand what they replace - the monolith. So far, everything we have built is a monolith. That just means one single application, one deployable unit, with all the features bundled together inside it. And honestly, for starting out, that is perfectly fine. But as a system grows huge, the monolith starts to hurt. Let me show you the three pains.

Pain one - one failure can take everything down. If the payments part crashes badly, it can drag the whole application down with it, including the parts that were working fine. No isolation. Pain two - you cannot scale just one part. Say only the orders feature is getting hammered with traffic. To handle it, you have to duplicate the entire giant app, wasting resources on all the parts that were not even busy. And pain three - risky deployments. A tiny one-line change to the inventory feature means rebuilding and redeploying the whole massive application. So releases become slow and scary.

Picture one enormous ship carrying everything - if one engine fails, the whole ship stops, and you cannot make just one part of it faster.

Slide 15 - Microservices architecture: the solution

ON SCREEN

The same system split into independent services, each with its own database, and three benefit cards.

SAY (read this aloud)

So here is the alternative - microservices. The idea is to break that one giant app into many small, independent services. Each one owns a single business job, has its own database, and they talk to each other over the network, usually with REST - the very thing you have been learning.

Now compare directly with the monolith's three pains. Independent deployment - you can update the payment service and ship just that one; the others keep running, untouched. Independent scaling - if orders is busy, you run more copies of just the order service; pay only for what you actually need. And failure isolation - if payments goes down, users and inventory carry right on serving. One failure no longer means total collapse.

Instead of one enormous ship, picture a fleet of small boats. If one has engine trouble, the others sail on, and you can speed up just the boats that need it.

DELIVERY TIP

Be honest about the trade-off, as the slide notes - it shows maturity and stops them over-using microservices: they add network calls, distributed data, and operational complexity. They shine for large systems and large teams. A small project is often better as a tidy monolith. Knowing WHEN to use each is the real skill.

Slide 16 - Horizontal vs vertical scaling

ON SCREEN

Two columns - vertical scaling (one bigger server) versus horizontal scaling (more servers behind a load balancer).

SAY (read this aloud)

Let's talk about handling more traffic - what we call scaling. There are two ways to do it, and the difference is simple once you picture it.

The first is vertical scaling - scaling up. This means taking your one machine and making it more powerful - more memory, more CPU. It is simple, but it has a ceiling - you cannot keep adding power forever, there is a biggest machine you can buy. And big machines get very expensive. Plus, it is still just one machine - one point of failure.

The second is horizontal scaling - scaling out. This means adding more machines, each running a copy of the same service, with a traffic director in front of them. There is no real ceiling - just keep adding machines. It is cost-effective, because you use many ordinary machines instead of one giant one. And it is naturally fault-tolerant - if one machine dies, the others carry on.

Think of it like a shop checkout. Vertical scaling is training your one cashier to work faster - there is a limit. Horizontal scaling is just opening more checkout counters.

EMPHASISE

Connect it forward to set up Session 4: microservices are designed for horizontal scaling. Because each service is small and remembers nothing between requests, you can spin up as many copies as you need and spread traffic across them - which is exactly how the cloud works.

Slide 17 - Load balancing & high availability

ON SCREEN

A load-balancer diagram: clients to a load balancer to three instances of a service.

SAY (read this aloud)

So if you have three copies of a service running, a natural question pops up - which copy does each request go to? That is the job of a load balancer. It sits in front of your copies and spreads the incoming traffic evenly across them. Think of it as the person at the front of a bank who directs you to whichever teller is free.

This gives us two big wins. First, load balancing itself - no single copy gets overwhelmed, because the work is shared. And second, high availability - this is the important one. If one copy crashes, the load balancer simply notices and stops sending traffic to it. The other copies keep serving, and your users never even notice there was a problem. The system stays up.

In the Spring world, this is handled by tools in Spring Cloud - things with names like Eureka and an API Gateway. Don't worry about those names yet - they are exactly what we build, hands-on, in Session 4. For now, just hold the picture: a traffic director sharing work across copies, keeping everything running smoothly.

Slide 18 - Full architecture at a glance

ON SCREEN

A microservices diagram with the Post service highlighted as 'what we built' and others greyed out.

SAY (read this aloud)

Here is the payoff slide - the one that puts everything in perspective. Look at this diagram of a full microservices system. There is a gateway at the front, the single entry point, and behind it sit many services - a post service, a user service, a payment service, and so on.

Now here is what I want you to really take in. That highlighted box - the Post service - is exactly the kind of thing you have built across Sessions 1 to 3. Your clean, layered API IS one of these services. All the greyed-out boxes are future territory, things you will build later in your career. But the highlighted one? You can build that today.

So the message is this: you are not just building toy apps to pass the time. You are building the fundamental unit of a real, large-scale system. A clean, layered, well-behaved service is exactly the piece that slots into a picture like this. Master that one service - which you are doing right now - and a giant system is really just many of them, cooperating. That should feel pretty good.

Recap & Close

Slide 19 - What's coming next

ON SCREEN

Four cards: Spring Security deep-dive, Spring Cloud basics, inter-service communication, database per service.

SAY (read this aloud)

Today you got teasers of security and microservices. Let me show you how those teasers become real skills in future sessions.

A Spring Security deep-dive - going past today's default login page to real authentication, with JWT tokens and role-based access, so different users can do different things. Spring Cloud basics - the actual tools, Eureka so services can find each other, and an API Gateway as the single front door. Inter-service communication - how one service calls another, building right on top of that RestTemplate idea from Session 2. And database-per-service - why each microservice keeps its own data, and how to manage that.

And the encouraging part is - everything connects. The clean, layered service you are learning to build today is the foundation that every single one of these advanced topics sits on top of. You are building the base, and the base is the hard part.

Slide 20 - Session recap + annotation cheat-sheet

ON SCREEN

Five plain-English concepts at the top, an annotation grid below: @Entity, @RestController, @Repository, @Service, @ResponseStatus, @Autowired.

SAY (read this aloud)

Let's recap today in plain English - five key ideas. One, a DTO is a simple class that carries only what the caller needs, keeping your entity safely hidden. Two, layered architecture is Controller to Service to Repository, each with one job. Three, a custom exception is your own error class with a meaningful message. Four, at-ResponseStatus maps that exception to the right code, like 404, automatically. And five, orElseThrow is the clean way to throw when something is not found - no messy null checks.

And the annotation grid below is your revision anchor. At-Entity marks a database table. At-RestController marks the web layer that handles HTTP. At-Repository marks the data layer. At-Service marks the business layer. At-ResponseStatus sets the status code. And at-Autowired injects a helper. Keep this slide handy when you revise.

Slide 21 - Closing

ON SCREEN

The closing blue slide: 'Let's build it.'

SAY (read this aloud)

And that is the theory for Session 3. You can now explain why we split an app into layers, why we never expose our entities directly, and how a couple of small pieces give callers clean, friendly error messages. That is genuinely professional-level understanding.

Now let's open the lab and build the layered Post API together - the entity, the DTO, the custom exception, the service with its two little mapper methods, and the controller on top. By the end, you will have a clean, professional, persistent API that you built yourself. Let's build it.

DELIVERY TIP

Move into the lab while the 'rough to professional' narrative is fresh. Remind them the lab follows the exact build order from the layered-architecture slide, so that diagram is their map.